We're almost done with our app. In the next chapter, we'll use an asynchronous job to update the status of our SMS message, so that it doesn't say "queued" in the UI.

Wrapping Up

HTTP clients are relatively easy libraries to use, but a lot of complexity hides beneath the surface. The varying levels of abstraction can create a bit of confusion regarding which library you should use for your app. Libraries at the lower levels of abstraction (such as socket protocols or pooled connections) are usable but will usually leave you wishing that they had more features. Full-featured libraries such as Req and Tesla will provide a solid foundation for any app out there. It's largely a matter of preference, but we picked Req for our app.

It's easy to use Req to make HTTP requests, but you'll likely need a configuration that tells the system which URL domain to send requests to, which authentication to use, and so on. You can store all of your application-specific configuration inside of the `config` directory, which is automatically loaded into your application. You can even set up a secret configuration such as `dev.secret.exs`, which you would then exclude from version control.

Req provides top-level functions such as `Req.post!/2` and `Req.get!/1` that make it a breeze to issue HTTP requests. But you can wrap up commonly used options into a base configuration with the `Req.new/1` function. Plus, if your app has more advanced needs, you can customize the entire request and response lifecycle of Req. We were able to use our Req-based API in the `send_sms_message/1` function without changing the rest of our app because we initially built it in a mocked-out way in the previous chapter.

Our last application-oriented chapter is up next. We'll be looking at asynchronous jobs in Elixir and specifically diving into the Oban library.